SE 4485: Software Engineering Projects

Spring 2026

Final Report

Group Number	Group 11
Project Title	Solution to Translate Auditd and Syslog into JALoP
Sponsoring Company	Everfox
Sponsor(s)	Dustin Endres
Students	1. Alliya Ahmad (Group Leader) (dal548178) 2. Andrew J Hoberer (ajh220009) 3. Albion Tony Krasniqi (atk210001) 4. Caleb Smitheart (rcs220002) 5. Tanner R Raley (trr220002) 6. Arav Sibal (axs220473) 7. Jennah Shahein (jxs220148) 8. Laurenne Oliver (lxo230001)

Executive Summary

This project presents a solution developed for Everfox to translate Linux auditd and syslog log data into the JALoP 2.x format. Modern Linux environments depend significantly on logging facilities like rsyslogd and auditd for auditing, monitoring, and logging purposes. Nevertheless, enterprises implementing the JALoP protocol often encounter interoperability problems since their log data is not directly compatible with JALoP-enabled systems.

The purpose of this assignment was to create a plugin module able to transform native Linux logs into valid JALoP XML output without compromising system performance or security. The proposed solution is a hybrid software/hardware component that seamlessly interacts with existing rsyslogd logging services without altering their core operations, and it can save output either locally or remotely.

The development process adhered strictly to the classic waterfall software engineering methodology and went through stages of requirements analysis, architecture design, detailed design, implementation, and testing. The end product contains two major components: a JALoP Translator Plug-in and a JALoP Reader subsystem. Our solution focused on usability by our Everfox sponsors as well as a strong integration into the existing JALoP body of work in an effort to promote the open source software ecosystem.

The team conducted extensive testing to validate system functionality in adherence to the specifications provided by Everfox. The completed project allows organizations to adopt the JALoP standard while continuing to use existing Linux logging infrastructure by simply implementing our plugin to the end of their existing systemd pipelines.

Table of Contents:

Executive Summary **2**
Table of Contents: **3**
List of Figures **3**
List of Tables **3**
1. Introduction **3**
1.1. Purpose and Scope 3
1.2. Product Overview (including purposes, capabilities, scenarios for using the product, etc.) 4
1.3. Structure of the Document 4
1.4. Terms, Acronyms, and Abbreviations 5
2. Project Management Plan **6**
2.1. Project Organization 6
Responsible for creating test cases/running tests and managing documentation **6**
Role Interaction: **7**
2.2. Lifecycle Model Used 7
2.3. Risk Analysis 7
2.4. Software and Hardware Resource Requirements 8
2.5. Deliverables and Schedule 9
2.6. Monitoring, Reporting, and Controlling Mechanisms 9
2.7. Professional Standards 9
2.8. Evidence all the artifacts have been placed under configuration management 10
2.9. Impact of the project on individuals and organizations 12
3. Requirement Specifications **12**
Actors 13
Main Use Cases 13
Explaining the Use Case Relationships: 14
1. Performance and Efficiency 18
2. Reliability and Availability 18
3. Security 18
4.. Maintainability and Portability 19
5. Usability (Configuration) 19
4. Architecture **19**
5. Design **21**
6. Test Plan **26**
7. Evidence the Document Has Been Placed under Configuration Management **28**
8. Engineering Standards and Multiple Constraints **29**
9. Additional References **29**

Acknowledgment.....30

List of Figures

- Figure 1: Graphic Use Case Model 16
- Figure 2: Architectural Model of the JALoP Translator System 21
- Figure 3:Static model (Class Diagram) 23
- Figure 4: Dynamic Model (Sequence Diagram) 24

List of Tables

- Table 1: Terms, Acronyms, and Abbreviations 5
- Table 2: Risk Analysis 8
- Table 3: Project Management Plan Version History 11
- Table 4: Architecture Documentation Version History 11
- Table 5: Detailed Design Documentation Version History 12
- Table 6: Test Plan Version History 13
- Table 7: Functional Requirement Traceability to Detailed Design 25
- Table 8: Traceability of Test Cases to Use Cases 28
- Table 9: Final Report Configuration Management Version History 29

1. Introduction

1.1. Purpose and Scope

This project was designed to create a system, capable of translating auditd and syslog messages from Linux operating system to JALoP 2.x XML schema and delivering them to Everfox. The system allows to incorporate JALoP workflow into existing organization's processes without changing the existing Linux log architecture. The main objectives of the project involve the development of a plugin-based translation system, which will translate native Linux log messages into JALoP XML messages, deliver translated messages via network interfaces, store them on hard disk drives, validate generated JALoP messages and ensure reliable processing of the invalid ones. It should be noted that the system must work on x86_64 RedHat Linux environment in conjunction with rsyslogd. The project was executed taking into account limitations on resource consumption, offline operation, modularity of components, compatibility with current Linux log system and adherence to engineering and security standards. The solution was required to operate in background mode without user intervention.

1.2. Product Overview (including purposes, capabilities, scenarios for using the product, etc.)

The developed product is a translation and validation framework that translates Linux syslog and auditd logs into the JALoP 2.x format. The application consists of two separate subsystems – the translation plugin and the JALoP Reader subsystem. The translation plugin is

implemented as an external plugin running alongside rsyslogd and communicating with the JALoP Reader subsystem that performs the actual validation and storage of the logs.

The system has a number of features:

- Conversion of Linux native auditd and syslog logs to JALoP-compliant XML
- Local storage and network routing options for the translated logs
- JALoP validation and verification of the generated logs
- Processing of invalid and malformed logs
- Efficient performance allowing the tool to be used in enterprise RedHat Linux deployments
- Modularity of subsystems to enhance maintainability and expandability in the future

The key scenario of using the solution implies the availability of an existing Linux system that uses rsyslogd and auditd as event generators. The translator plugin processes the finished log entries, converts them to the JALoP format, and passes the results to the JALoP Reader subsystem where validation is performed.

1.3. Structure of the Document

The document will have several primary sections, which shows the development process and implementation of JALoP translation system.

Section 1 includes Introduction, which gives the detailed description of the purpose and scope of the project, product description, document outline and definitions/acronyms used.

Section 2 contains information about the Project Management Plan including team structure, lifecycle, risk assessment, scheduling, and professional standards used.

In Section 3 you can find the Description of System Requirements that involve stakeholders involved, use-case models, functional requirements, and non-functional ones.

The description of the system architecture can be found in section 4, which covers the aspects of the architectural style, technologies used, rationale, and requirement traceability.

The Detailed Design is described in section 5 and includes class and sequence diagrams, design rationale. Section 6 contains information about the Test Plan, which covers test cases, testing techniques, and traceability from tests to requirements.

Configuration management processes applied in the project are mentioned in section 7.

Finally Section 8 contains the Engineering Standards and Constraints used while developing the product.

1.4. Terms, Acronyms, and Abbreviations

Term / Acronym	Definition
JALoP	Journal of Audit Log Protocol an open standard for securely transferring and storing audit log data
JALoP 2.x	Version 2 of the JALoP specification; the target XML output format for this project
auditd	Linux Audit Daemon — a user-space component that records security-relevant events on a Linux system
syslog	A standard protocol and service for message logging on Unix/Linux systems
rsyslogd	Rocket-fast System for Log processing — an enhanced syslog daemon used on Linux; the pipeline host for this project's plugin
XML	Extensible Markup Language — the structured data format used for JALoP output
x86_64	A 64-bit processor architecture (also called AMD64); the target hardware platform for this project
RedHat Linux	A Linux distribution by Red Hat; the target operating system environment
Plugin	A software component that integrates into an existing system to extend its functionality without modifying the core system
API	Application Programming Interface — a defined interface for software components to communicate with one another
CM	Configuration Management — the practice of tracking and controlling changes to software artifacts throughout development
IEEE	Institute of Electrical and Electronics Engineers. Standards body referenced for project management (IEEE Std 1058)
IEEE Std 1058	IEEE Standard for Software Project Management Plans. It is used to guide the structure of this project's management plan
SE	Software Engineering
systemd	A Linux init system and service manager; the pipeline infrastructure into which the translator plugin integrates
FOSS	Free and Open Source Software. It is a software licensed to allow free use, modification, and redistribution
Everfox	The sponsoring company for this project; provides requirements and end-user deployment context

JALoP Reader	The subsystem responsible for validating and storing translated JALoP log entries
Translator Plugin	The subsystem that converts native Linux auditd and syslog logs into JALoP-compliant XML
JALoP XML Schema	The formal XML schema definition that governs valid JALoP record structure and content

2. Project Management Plan

2.1. Project Organization

- **Project Manager (1) - Alliya Ahmad**
  Coordinates the team and serves as the main point of contact with Everfox.
- **Technical Lead (1) - Tanner Raley**
  Owns the technical direction and facilitates architecture helping with major design decisions.
- **Backend Developers (4) - Laurenne Oliver, Caleb Smitheart, Alliya Ahnad, Albion Tony Krasniqi**
  Builds translation logic, parsing and core features
- **Research and Standards Analysts (2) - Arav Sibal, Andrew Hoberer**
  Research JALoP, syslog, and auditd formats and define how everything maps together.
- **Testing & Documentation Lead (1) - Jennah Shahein**
  Responsible for creating test cases/running tests and managing documentation

Role Interaction:
The Project Manager coordinates task assignments and schedules meetings based on project priorities. The tech lead works with backend developers to make design and implementation decisions. Research and standards analysts provide specifications and mapping rules that inform development. The testing and documentation lead organizes the testing effort, verifying functionality of the product and producing documentation describing it. All roles communicate regularly in the interest of strong cooperation and conflict resolution.

2.2. Lifecycle Model Used

We used the Waterfall Model for this project. The Waterfall model follows a sequential development process in which each phase must be completed before the next phase begins which aligns with the deadlines we have on the course website. Some key phases of the waterfall model include planning, requirements analysis, system architecture, detailed design, implementation, testing, and final delivery.

The reason why we chose this model is because of the fixed and sequential deadlines of the project deliverables and hard submission deadlines defined by the course structure and website. Each phase requires us to submit formal documentation that must be completed and reviewed before going on to the next phase. For example the Project Management Plan is submitted first, then the Requirements Documentation, then the Architecture Documentation, and then the Detailed Design Documentation and so on. Another reason why we chose the Waterfall Model is that the project requirements are unlikely to change after they are defined. This model works well both due to the SE4485 course requiring documentation and because the logging system has to meet required logging/auditing standards for compliance reasons as stated in the Everfox Slides.

2.3. Risk Analysis

Based on IEEE Std 1058 guidance, the project team identified potential risks, evaluated their likelihood, and defined strategies to reduce their impact on the project schedule and technical outcomes.

Risk Name	Likelihood	Severity	Mitigation Strategy
Limited Familiarity with the JALoP Standard	High	Low	The team spent time early in the project reviewing the JALoP specification and exploring the open-source JALoP repository. Small prototypes will be developed to help

			build understanding.
Integration issues between syslog/auditd and JALoP	Medium	High	The translation solution was developed in stages, with regular testing to confirm that the output meets JALoP requirements. Sponsor feedback was requested at key points.
Schedule constraints due to academic workload	Medium to High	Medium	Tasks will be divided among team members; milestones will be clearly defined; buffer time will be included in the schedule.
Delays caused by dependence on sponsor feedback	Low	Medium	The team prepared questions in advance and development was based on documented assumptions when immediate feedback was not available.

2.4. Software and Hardware Resource Requirements

- The developed software needs to be able to run on a RedHat Linux enterprise server, as that is the environment the end user intends to use the developed software in.
- The development and testing CPU architecture will be x64_86, as that is also the architecture of the end user's server.
- The software should not need an internet connection to function properly, all logic should be handled locally, using predownloaded libraries if necessary.
- The software should not build off of any code using any sort of copyleft licensing. Any existing code used should be free and open source, as this software will later be used in a for-profit, closed source system.

All of the above requirements represented new technologies for our team members as this is our first time contributing a plugin to an existing system. One surprising software we became intimately familiar with was Git and Github. Given that we had to read through and tinker with a lot of existing codebases for the open standard JALoP, we became intimately familiar with the git activities like cloning and pull requests that come with contributing to open source software.

2.5. Deliverables and Schedule

Everfox has defined two major deliverables for this project; a mid-term progress check and a final project presentation. Both consist of a Teams meeting with all project shareholders where our team will present all relevant research, share what we've learned, and provide a demonstration of the project's executable.
Our team has also defined several intermediate deadlines to ensure sufficient progress. These deliverables are as follows:
2/11 - Conclude initial research on JaLoP, Syslog, and Auditd
2/13 - Local environment setup for local testing
3/11 - Progress check with Everfox
3/18 - Conduct internal reflection on status of the project using Everfox feedback, formulate a plan on high priority areas to proceed on.
5/6- Final product presentation for Everfox

2.6. Monitoring, Reporting, and Controlling Mechanisms

Weekly Status Reports will summarize progress on product development and any issues hindering development. Code reviews through GitHub to ensure adherence to the Everfox provided product specifications. Attendance forms to track team members participation.
Another form of project control was the different deliverables required for the course such as Project Management Plan, Requirements Documentation, Architecture Documentation, Detailed Design Documentation, Test Plan and Final Project Report.

2.7. Professional Standards

- On the first occurrence of unacceptable behavior, we plan to resolve the problem, and document the event in the Teams meeting.
- On the second occurrence, notify both the TAs and the instructor of the current situation and any/all past events. Work out a solution to proceed forward that resolves for all parties.
- On the third occurrence, again notify the TAs and the instructor of the current problem. A meeting will then take place between the instructor, TAs, team leader, and the team members involved in the conflict.

- Definition of unacceptable behavior:
    - Missing meeting with Everfox for any reason
    - Missing weekly meeting without prior notice of legitimate reason
    - Not completing work of good quality prior to deadline
    - Failure to communicate with teammates for progress checks

2.8. Evidence all the artifacts have been placed under configuration management

All project artifacts were maintained under configuration management using Google Docs version history. All modifications require peer review and approval from at least two team members prior to acceptance.

Below are the configuration management histories for each of our project artifacts:

Project Management Plan:

Version	Date	Description	Authors
1.0	2026-02-03	Initial draft based on class template.	Jennah Shahein
1.1	2026-02-04	Filled in the entire document during a group work session.	Entire Team

Requirements Documentation:

Name	Version Before	Version After	Difference between 2 consecutive versions	Review	Comments
Andrew	0.00	0.01	Document created	Everyone ship-it	
Laurenne	0.01	0.02	CM evidence section completed	Everyone ship-it	
Caleb	0.02	0.03	Use Case Section completed	Everyone ship-it	
Jennah	0.03	0.04	Introduction and table of contents	Everyone ship-it	
Alliya	0.04	0.05	List of Figures and Tables	Everyone ship-it	

Architecture Documentation:

Name	Version Before	Version After	Difference between 2 consecutive versions	Review	Comments

Jennah	0.00	0.01	Document created	Everyone ship-it	
Caleb	0.01	0.02	Architectural style and model discussion completed	Everyone ship-it	
Laurenne	0.02	0.03	Introduction and table of contents completed	Everyone	
Albion	0.03	0.04	Added Traceability for requirements	Everyone	
Arav	0.04	0.05	Abstract Technology/Hardware module completed	Everyone	

Detailed Design Documentation:

Name	Version Before	Version After	Difference between 2 consecutive versions	Review	Comments
Jennah	0.00	0.01	Document created	Everyone ship-it	
Arav	0.01	0.02	Introduction, Abstract	Everyone ship-it	
Tanner	0.02	0.03	GUI	Everyone ship-it	
Caleb	0.03	0.04	Design Class Diagram, Sequence Diagram and their rationales	Everyone ship-it	
Laurenne	0.04	0.05	Added List of Figures and List of Tables	Everyone ship-it	
Jennah	0.05	0.06	Added the requirements traceability table. Table of contents	Everyone ship-it	

Test Plan:

Name	Version Before	Version After	Difference between 2 consecutive versions	Review	Comments
Andrew	0.00	0.01	Document created	Everyone ship-it	
Caleb	0.01	0.02	Added test set, techniques and table	Everyone ship-it	
Andrew	0.02	0.03	Introduction, table of contents, and formatting	Everyone ship-it	

Codebase:

Configuration management done using and git and github, seen here:
https://github.com/RTanner18/JALoP-Translator

2.9. Impact of the project on individuals and organizations

This project will be useful for our sponsors at Everfox that requested this data to make their workflows more efficient, and from a larger scope, it furthers the cause of open sourcing useful software. Because we created our project with open licensing in mind from the outset, as was dictated by Everfox, all aspects of our design incorporate teachings from the Free and Open Source Software movement. In supporting the FOSS movement, we provide free software allowing for those without economic access to learn about the syslog and JALoP logging formats, while also providing a societal good in promoting increased transparency in software, which allows for better consumer understanding of the products they use as well as allows for better security derived from public scrutiny.

3. Requirement Specifications

3.1. Stakeholders for the system

The stakeholders for the JALoP Translation Project are basically everyone that is involved in developing, using, supporting, or benefiting from the system which includes us the 8 team members as well as Dustin, our sponsor, and other Everfox employees.

1. Everfox: Everfox was our project sponsor and they provided us with system requirements as well as the problem that we needed to solve that would help them in their business.

   Within the Everfox team theres a few different stakeholders as well based on the

different teams:

1. System Administrators: We want the system to allow Everfox system admins to configure rsyslog, manage log output destinations, and monitor the system during operation.
2. Security Analysts: Use the generated JALoP/JAF logs for monitoring, auditing, and security event investigation.
3. Developers: Design, implement, test, and maintain the JALoP and JAF output modules.
4. Audit and Compliance Teams: Depend on accurate and reliable log data for auditing and compliance purposes.

2. Dustin (Project Sponsor/Advisor): Dustin met with us weekly and made sure we were developing the project in the way that it was needed and helped us figure out the project scope as well as pacing and technical expectations.

3. Our 8 group members, the team, developing the project

4. rsyslog Open-Source Community: Provides the plugin framework and API standards that the project is built on.

3.2. Use case model for functional requirements

The use case model basically shows how different users and system components interact with the rsyslog to JALoP conversion system we built. The purpose of the system is to take rsyslog data and then convert it into JALoP/JAF XML format so it is usable and either store it locally or send it over the network. It also includes basic validation we created through test cases to make sure the generated output is correct.

Actors

1. rsyslogd Output: Sends log data into the system for processing

2. Admin: Reviews and verifies the generated JALoP output

3.Network Interface: Handles sending formatted log data across the network

4. OS Filesystem: Handles writing formatted log data to local storage.

Main Use Cases

1.Translate rsyslog data into JALoP format

2.Send JALoP formatted data over the network

3.Store JALoP formatted data locally

4.Verify the generated JALoP output

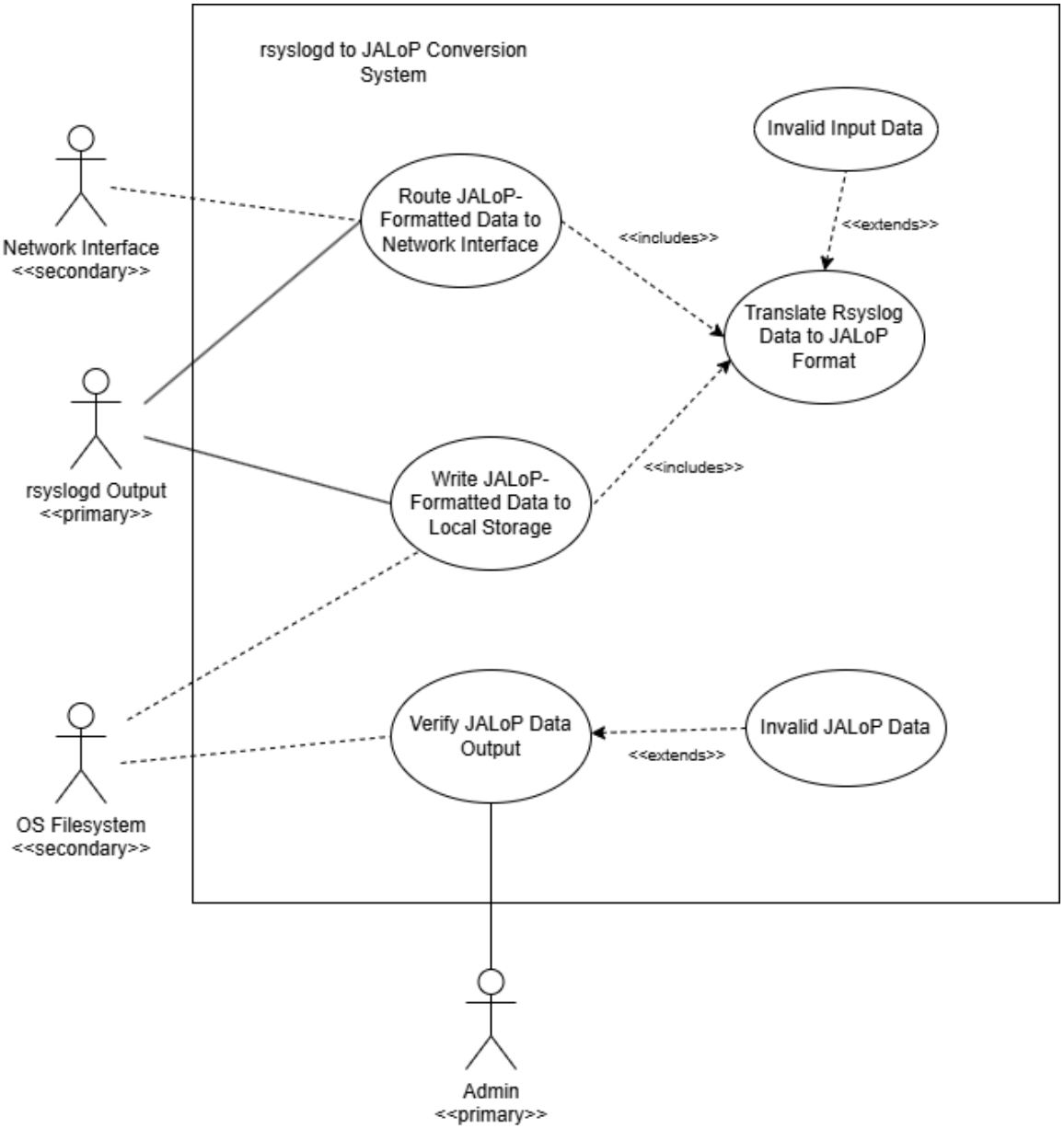
5.Handle invalid or corrupted input data

6.Detect invalid JALoP output during verification

Explaining the Use Case Relationships:

The system has to translate rsyslog data before it can either send it over the network or store it locally. The use case model also includes exception cases like invalid input data or invalid JALoP output to show how the system handles errors without interrupting the main logging process.

3.3. Graphic use case model



3.4. Textual Description for each use case

Name: Translate RsyslogData to JALoP Format	
Actors Involved	Primary actors: Rsyslogd Service
Preconditions	1. Some event has occurred on the system 2. rsyslogd daemon has finished processing the event internally
Main Flow	1. rsyslogd passes the completed log to the translation system plugin 2. The system reads through the passed log Extension Point: Invalid Input Data 3. The system translates the parsed log into the JALoP log format

Postconditions	The system has an internally loaded translation of the input log into the JALoP format
Exceptions	The parsed data is of an incorrect format
Special Requirements	N/A

Name: Invalid Input Data	
Actors Involved	Primary actors: rsyslogd Service
Preconditions	The system found invalid log data while parsing
Main Flow	1. The system throws an error 2. The log translation fails 3. A log for the translation failure is created instead
Postconditions	1. The system has finished executing 2. An error log has been successfully output
Exceptions	N/A
Special Requirements	N/A

Name: Route JALoP-Formatted Data to Network Interface	
Actors Involved	Primary actors: rsyslogd Service Secondary Actors: Network Interface
Preconditions	1. Some event has occurred on the system 2. rsyslogd daemon has finished processing the event internally
Main Flow	1. Include (Translate rsyslogData to JALoP Format) 2. The system initializes a socket 3. The system transmits the formatted log 4. The system closes the socket
Postconditions	1. The system has finished executing 2. The log has been successfully transmitted to the network interface
Exceptions	N/A
Special Requirements	N/A

Name: Write JALoP-Formatted Data to Local Storage	
Actors Involved	Primary actors: rsyslogd Service Secondary Actors: OS Filesystem
Preconditions	1. Some event has occurred on the system 2. rsyslogd daemon has finished processing the event internally
Main Flow	1. Include (Translate rsyslogData to JALoP Format) 2. The system writes the translated log to a log file
Postconditions	1. The system has finished executing 2. The log has been successfully written the log to a local file
Exceptions	N/A
Special Requirements	N/A

Name: Verify JALoP Data Output	
Actors Involved	Primary actors: Admin Secondary Actors: OS Filesystem
Preconditions	The system has finished writing a translated log to a file
Main Flow	1. The admin or automatic verification triggers a check of the JALoP logs 2. The system validates the translated log file against proper JALoP syntax Extension Point: Invalid JALoP Data 3. The system reports that the log has been validated successfully.
Postconditions	1. The system has finished executing 2. The log file has been successfully validated.
Exceptions	The file contains incorrectly formatted or invalid JALoP log data
Special Requirements	N/A

Name: Invalid JALoP Data	
Actors Involved	Primary actors: Admin
Preconditions	1. The JALoP reader section of the system found invalid JALoP log data
Main Flow	The system reports that the log data was found to be invalid.
Postconditions	1. The system has finished executing

	2. An alert has been made about the invalid log file.
Exceptions	N/A
Special Requirements	N/A

3.5. Rationale for your use case model

This use case model was adapted from the requirements document we received from Dustin Endres detailing the project's requirements.

3.6. Non-functional requirements

We divided the nonfunctional requirements into 5 categories which are performance and efficiency, reliability and availability, security, maintainability and portability, and lastly usability. We will explain these nonfunctional requirements in more detail below.

1. Performance and Efficiency

Throughput: The JALoP and JAF output modules should be able to handle log messages fast enough that the rsyslogd main queue does not overflow during normal system usage.

Latency: Converting rsyslogd's native log format into JALoP/JAF XML should not add a lot of delay and we are targeting staying under 10 ms per message so logging is real time.

Resource Usage: The plugins should use minimal system memory and resources so they don't affect the performance of the host Linux system.

2. Reliability and Availability

Data Integrity: The JALoP module should make sure log data stays accurate and unchanged while being converted from the sources (systemd-journal or auditd)

Network Issues: When logs are sent over a network connection the JALoP module should handle basic network issues properly. So, for example, if a UDP packet is lost, the system should continue running normally without crashing or becoming unstable.

Fault Tolerance: If the optional JAF XML module fails it should not affect the main JALoP output module or interrupt the normal operation of rsyslogd..

3. Security

Access Control: This is to make sure no one who should not have access to the system should not. Local files created by the JAF or JALoP modules should use restricted file permissions like

600 or 640 so unauthorized users cannot access audit information. This is really important for a system with highly confidential information like the information Everfox works with.

Audit Compliance: The solution should work with auditd and audisp-syslog to make events are logged properly and the audit trail works.

4.. Maintainability and Portability

Standardization: We want to make sure the code can be used by any Everfox employee so the code should follow the rsyslog external plugin API standards from the official GitHub documentation.

Platform Constraint: The solution is designed specifically for x86_64 Linux systems.

Modularity: The JALoP and JAF modules should be built as separate plugins so our team can update, test, or debug them independently but also Everfox employees can use it easily as well.

5. Usability (Configuration)

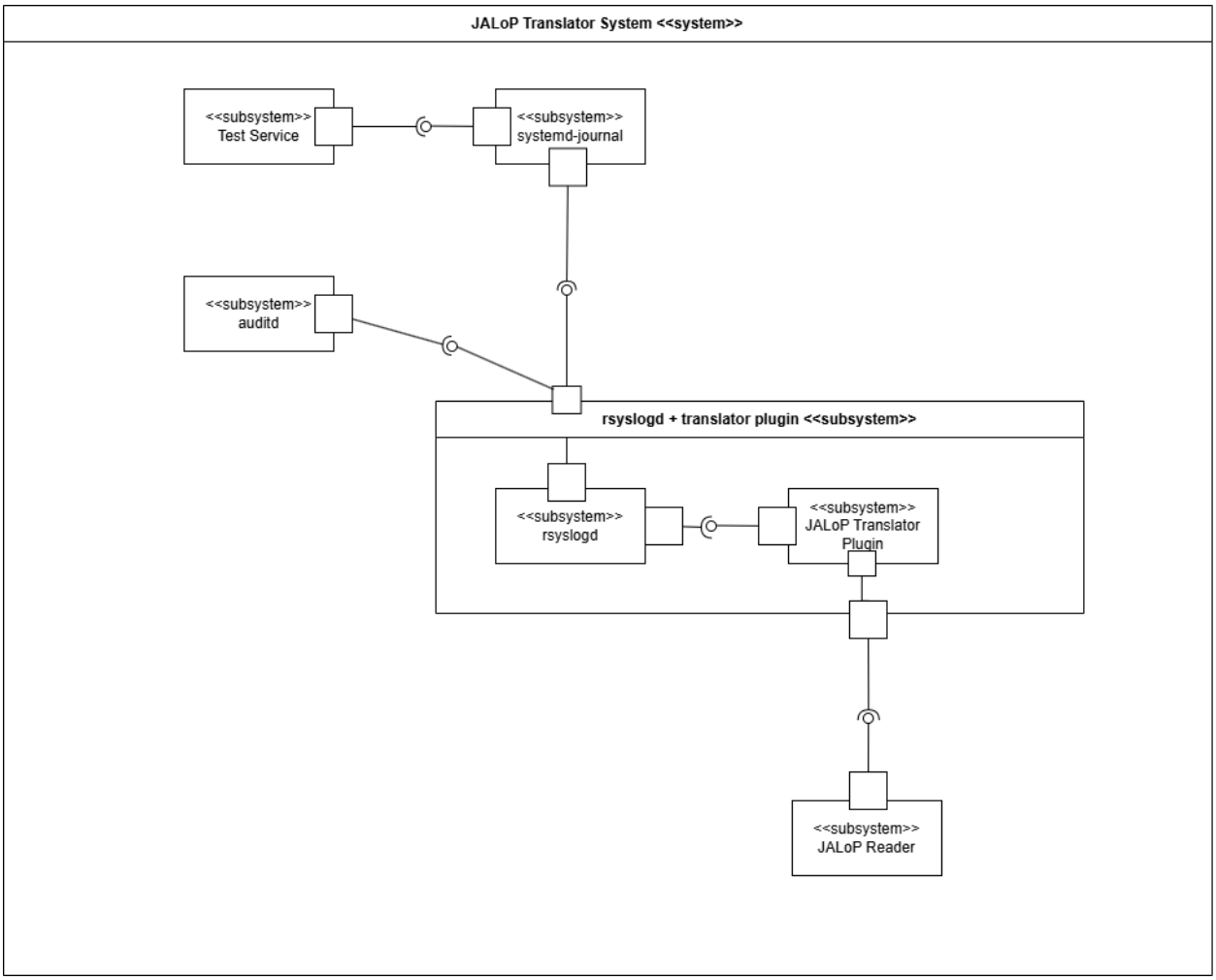
Configurability: Users should be able to change where the logs are sent which is either to a local file or over the network through rsyslog.conf or a plugin configuration file without having to recompile the code over and over again.

4. Architecture

4.1. Architectural style(s) used

Our system uses an object oriented style of architecture to support modularity and reusability. Different components of the system invoke methods from other components, treating them as a black box and avoiding any unnecessary coupling between object functionalities and features. The test service works independently of the rsyslog daemon, which itself is independent of the Jalop reader, fulfilling the features requested of the project by Everfox listed in their requirements specification.

4.2. Architectural model



4.3. Technology, software, and hardware used

The system is a plugin-based service that translates auditd and syslog logs (via rsyslogd) into the JALoP 2.x format. It is designed for seamless integration with existing logging setups on a RedHat Linux x64_86 server. The software must be built from free and open-source \code for compatibility with the final closed-source system. All translation occurs locally without internet access, and the resulting JALoP data can be saved to local file storage or sent via network-based output.

4.4. Rationale for your architectural style and model

The model closely follows from the requirements given to us by the Everfox team, but has been adapted into a more standard UML system-subsystem architecture diagram notation. The objected oriented architectural style was chosen because of its ability to cleanly separate the requested subsystems into objects, as well as its modularity. This system will eventually be reused with a real system and will take logs from non-test services, so the ability to change and swap in parts was essential to take into

consideration when making the choice, and an objected oriented style fulfilled that requirement nicely.

4.5. Traceability from requirements to architecture

This section maps each functional and non-functional requirement from the Requirements Documentation to the specific architectural components in the JALoP Translator System diagram. Where applicable, the corresponding use case from the Use Case Model is also referenced.

5. Design

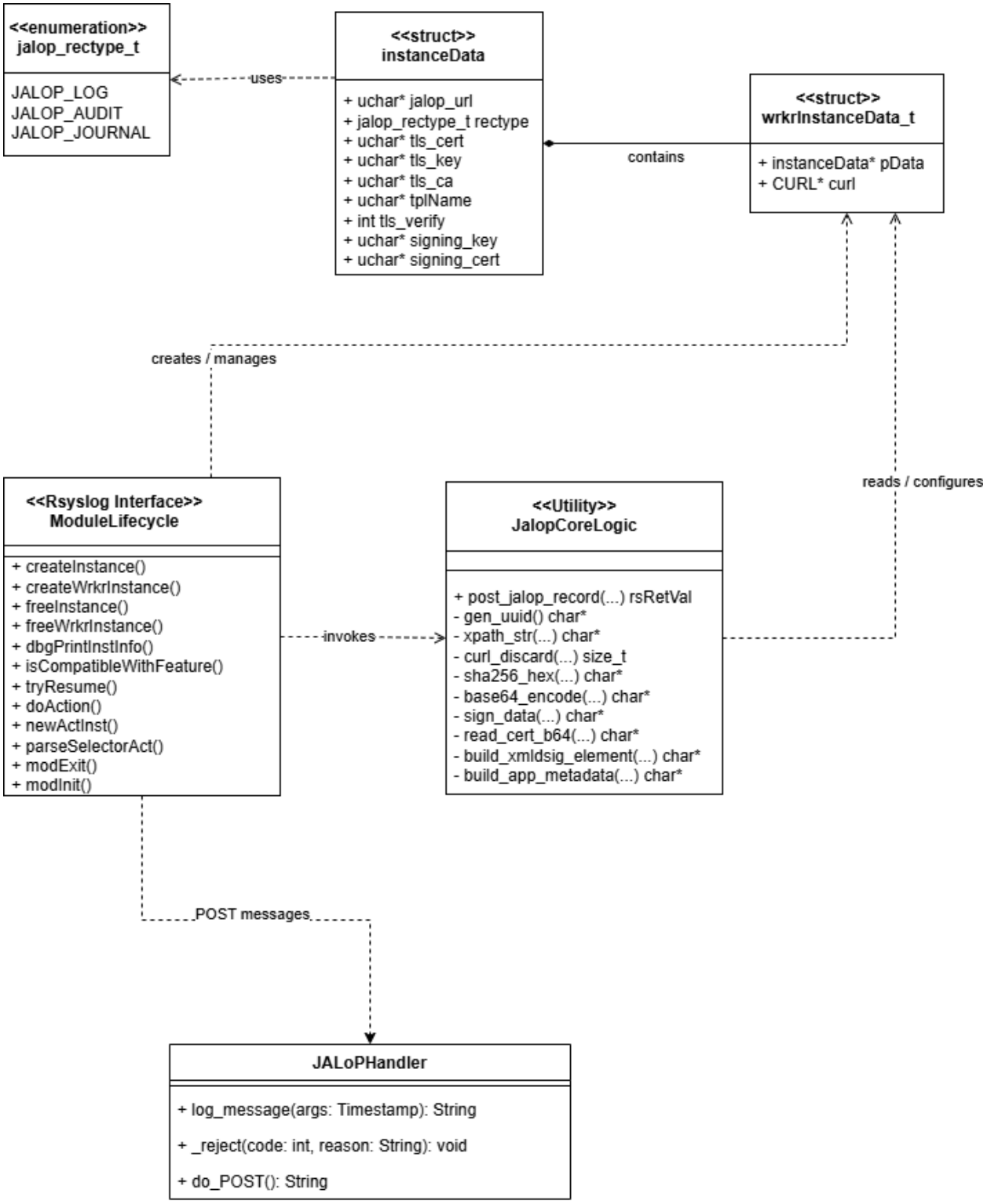
5.1. GUI (Graphical User Interface) design

Because our project is intended as a Command Line Interface per specifications, there is no traditional Graphical User Interface for the user to use. The entire system is designed to run and interact in the background with the Linux operating system, therefore no user input is required outside of project initialization and startup.

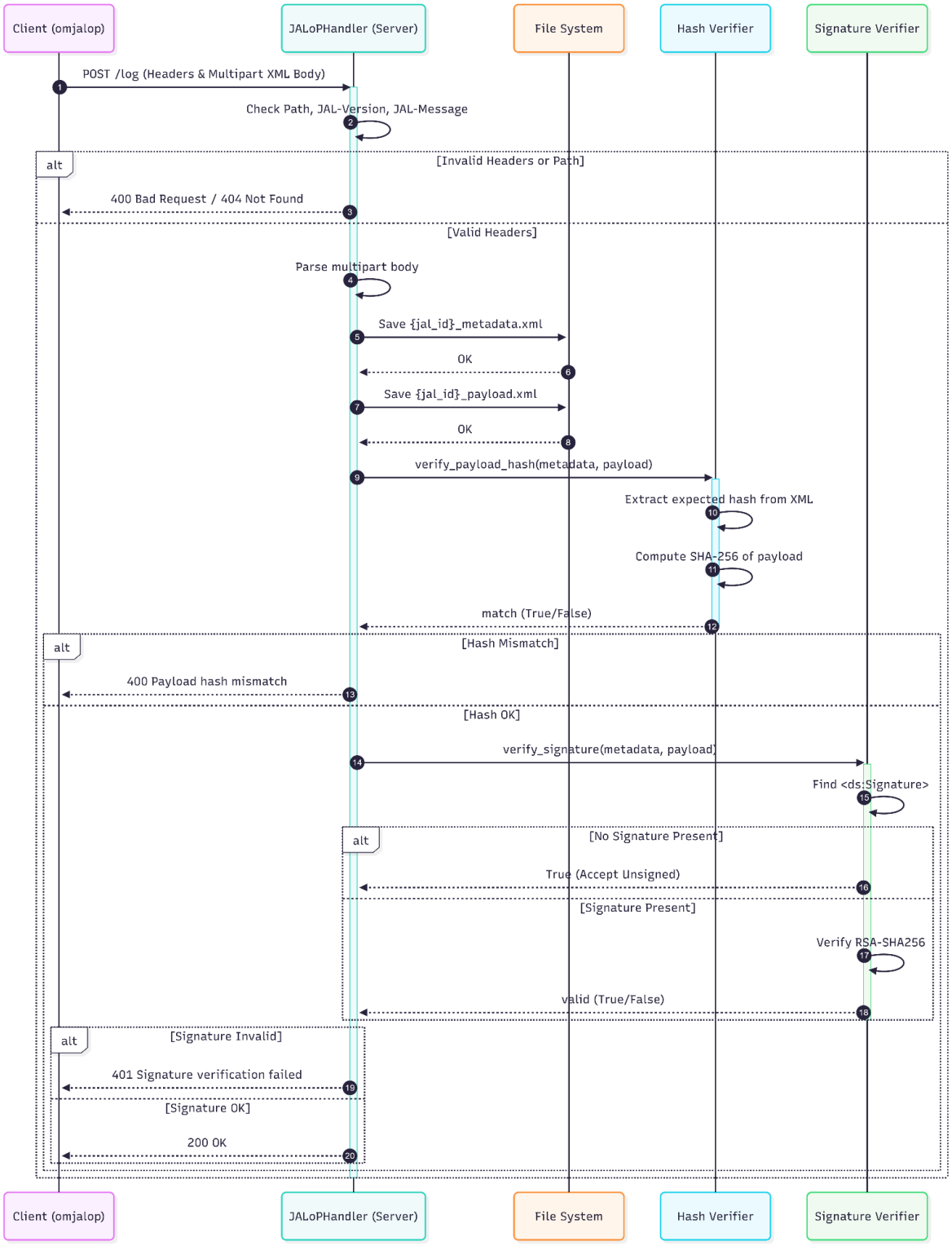
However, for testing the project we have created a simple Python script to translate the JALoP XML logs to a more readable format. Since its implementation, it has helped significantly with debugging and demonstrating the project progress to shareholders. An example of a log visualization can be found below.

That being said, eventually this script will be phased out as the project enters the production environment. Ideally it will be replaced by a JALoP compliant database implementation, which is out of scope for the purposes of this project.

5.2. Static model (Class Diagram)



5.3. Dynamic model (Sequence Diagram)



5.4. Rationale for your detailed design model

The system consists mainly of two distinct subsystems: the omjalop plugin for systemd, shown above in the first five classes in the design class diagram, and the receiver for these logs the

plugin sends over the network, shown above as the JALoPHandler class. The interactions between these two subsystems and other parts of the host environment are detailed above in the sequence diagram, when the plugin initiates the sequence by sending a systemd log, fully translated into the JALoP format, to the python receiver server. From there, the server parses the different parts of the log, saving its data to the local filesystem and verifying its associated hash.

5.5. Traceability from requirements to detailed design model

Requirement	ModuleLifecycle (systemd plugin)	JALoPHandler (JALoP Receiving Server)
FR 1: Route JALoP Formatted Data to Network Interface	ModuleLifecycle basically handles the overall lifecycle of the rsyslog plugin. It creates and manages the worker instances and triggers the core logic responsible for sending JALoP records over the network through JALoPCoreLogic and its post_jalop_record(...) function. We show this in the class diagram.	JALoPHandler is the receiving end point for incoming JALoP messages. In our sequence diagram we show the server accepting POST requests and then validating the headers and the path and finally returning HTTP responses like 200 OK, 400, and 401.
FR 2: Write JALoP Formatted Data to Local Storage	On the sending side, ModuleLifecycle prepares and passes translated log data for delivery, but it is not the primary local storage component in the receiving workflow. Its role is to package and forward the message.	JALoPHandler implements this requirement directly. The sequence diagram shows the handler parsing the multipart body and saving both {id_id}_metadata.xml and {id_id}_payload.xml to the file system before verification steps continue.
FR 3: Translate rsyslog data to JALoP format	ModuleLifecycle captures log events and performs the logic required to transform them. The class diagram shows it calling JALoPCoreLogic, which contains helper methods such as build_xmldsig_element(...), build_app_metadata(...),	JALoPHandler does not perform the original translation from rsyslog to JALoP instead, it receives the already formatted JALoP XML and processes it for validation and storage.

	gen_uuid(...), and signing/hash support needed to produce JALoP-compliant output.	
NFR 1: Performance and Efficiency	ModuleLifecycle supports performance by operating as an rsyslog plugin integrated into the Linux logging pipeline. The detailed design emphasizes efficient lifecycle management and direct invocation of posting logic rather than unnecessary intermediate steps.	JALoPHandler supports efficiency by performing a straightforward validation pipeline: check headers/path, parse multipart body, save files, verify hash, then verify signature. The sequence diagram shows a linear flow with early rejection on invalid input.
NFR 2: Reliability and Availability	ModuleLifecycle improves reliability by separating instance creation, worker creation, action parsing, initialization, and cleanup into defined lifecycle functions such as createInstance(), createWrkrInstance(), freeInstance(), and modExit(). This structured lifecycle reduces instability in the plugin.	JALoPHandler improves reliability by validating requests before accepting them and returning explicit error codes when failures occur. The sequence diagram shows handling for invalid headers/path, payload hash mismatch, and signature verification failure.
NFR 3: Security	ModuleLifecycle and JALoPCoreLogic address security through cryptographic support. The class diagram includes fields for TLS certificates/keys and signing material in instanceData, plus methods such as sha256_hex(...), sign_data(...), and Base64 encoding.	JALoPHandler enforces security by verifying payload integrity and checking signatures. The sequence diagram shows hash extraction from XML, SHA-256 computation, optional acceptance of unsigned content when no signature is present, and rejection when signature verification fails.
NFR 4: Maintainability and Portability	The plugin design is modularized into ModuleLifecycle, JALoPCoreLogic,	JALoPHandler is separated as an independent receiver subsystem, which improves maintainability by isolating server-side

	instanceData, wrkrInstanceData_t, and the jalop_rectype_t enum. This makes the design easier to change. The system is made for production grade RedHat Linux environments.	validation/storage concerns from plugin-side translation concerns. The rationale explicitly describes the system as two distinct subsystems.
NFR 5: Usability	There is no traditional GUI because the project operates as a command-line/background system. This supports usability for administrators by minimizing user interaction after start up.	JALoPHandler supports usability through clear server responses and straightforward processing behavior. We created a simple Python script to visualize JALoP XML logs in a more readable format for testing and demonstrations.

6. Test Plan

6.1. Requirements/specifications-based system level test cases

T = { t_1: Happy path test case, rsyslog event validly formatted, sent to local file,
t_2: Happy path test case, rsyslog event validly formatted, sent to network interface,
t_3: Invalid rsyslog data,
t_4: No data, invoke the function with no input
t_5: Garbage data, randomly generated sequence of bits
t_6: Verify JALoP output with correctly generated JALoP
t_7: Verify JALoP output with incorrectly generated JALoP
t_8: Verify JALoP output with garbage input
}

6.2. Techniques used for test generation

We used Boundary Value Analysis (BVA), a black box testing technique based on the requirements to generate our test cases. The quality was measured by simple line/block coverage, as the program is relatively small and does not branch frequently enough to warrant a more complex coverage criteria.

6.3. Assessment of the goodness of your test suite

We used simple code coverage to assess the goodness of our test suite, using the industry standard coverage.py to receive a coverage report for each of our test cases. The entire test suite provided a 100% code coverage, telling us we had no dead code and
Code coverage was deemed an accessible metric for determining the goodness of our test suite because, as stated in the previous section, the program is relatively small and not incredibly complex, so it doesn't require more rigorous goodness heuristics.

6.4. Traceability of test cases to use cases

Use Case → Test Case ↓	Route JALoP Formatted Data to Network Interface	Route JALoP Formatted Data to Local Storage	Translate Rsyslog Data to JALoP Format	Invalid Input Data	Verify JALoP Data Ouput	Invalid JALoP Data
Test Case 1		Is the expected result of test case	Necessary to complete the expected result of the test case			
Test Case 2	Is the expected result of test case		Necessary to complete the expected result of the test case			
Test Case 3				Is the input to the system specified by the test case		
Test Case 4				Is the input to the system specified by the test case		
Test Case 5				Is the input to the system specified by the test case		
Test Case 6					Is the expected result of test case	
Test Case 7						Is the input to the system specified by the test case
Test Case 8						Is the input

						to the system specified by the test case

7. Evidence the Document Has Been Placed under Configuration Management

The Final Report is maintained under configuration management using Google Docs version history. The document records authorship, timestamps, and differences between revisions. All modifications require peer review and approval from at least two team members prior to acceptance.

The Configuration Management Tool in use is Google Docs (built-in version history and change tracking).

Table 1: Configuration Management Version History

Name	Version Before	Version After	Difference between 2 consecutive versions	Review	Comments
Jennah	0.00	0.01	Document created		
Caleb	0.01	0.02	Added the project management section	Everyone ship-it	
Andrew	0.02	0.03	Added test plan and design	Everyone ship-it	
Jennah	0.03	0.04	Added introduction	Everyone ship-it	
Alliya	0.04	0.05	Working on Section 4	Everyone ship-it	
Jennah	0.05	0.06	Added Summary	Everyone ship-it	
Albion	0.06	0.07	Added traceability from requirements to architecture	Everyone ship-it	
Jennah	0.07	0.08	Table of contents	Everyone ship-it	

Arav	0.08	0.09	Standards/References	Everyone ship-it	
Alliya	0.09	0.10	List of Figures/Tables	Everyone ship-it	

8. Engineering Standards and Multiple Constraints

- IEEE Std 830-1998: Software Requirements [pdf]
- IEEE Std 29148: Requirements Engineering [pdf]
- IEEE Std 829-1983: Software Testing [pdf]
- ISO/IEC/IEEE Std 29119-1-(Revision-2022): Part 1 - Software Testing General Concepts [pdf]
- ISO/IEC/IEEE Std 29119-2-(Revision-2021): Part 2 - Test Process [pdf]
- ISO/IEC/IEEE Std 29119-3-(Revision-2021): Part 3 - Test Documentation [pdf]
- ISO/IEC/IEEE Std 29119-4-(Revision-2021): Part 4 - Test Techniques [pdf]
- IEEE Std 1058-1998: Software Project Management Plans [pdf]
- PMBOK® Guide: Project Management Body of Knowledge [pdf]
- IEEE Std 12207: Software Life Cycle Processes [pdf]
- IEEE Std 15939: Measurement Process [pdf]
- IEEE Std 1471-2000: Software Architecture [pdf]
- ISO/IEC/IEEE Std 42030:2019: Software, Systems and Enterprise Architecture Evaluation Framework [pdf]
- IEEE Std 1016-1998-(Revision-2009): Software Design [pdf]

9. Additional References

- Bass, L., Clements, P. and Kazman, R. (2003). *Software Architecture in Practice*. Addison-Wesley.
- Humphrey, W.S. and Thomas, W.R. (2010). *Reflections on Management: How to Manage Your Software Projects, Your Teams, Your Boss, and Yourself*. Pearson Education.
- Hyman, B. (1998). *Fundamentals of Engineering Design*. New Jersey: Prentice Hall.
- Jorgensen, P.C. (2013). *Software Testing: A Craftsman's Approach*. Auerbach Publications.
- Lamsweerde, A.V. (2009). *Requirements Engineering: From System Goals to UML Models to Software Specifications*. John Wiley.
- Larman, C. (2012). *Applying UML and Patterns: An Introduction to Object Oriented Analysis and Design and Iterative Development*. Pearson Education.
- Larson, E. and Gray, C. (2014). *Project Management: The Managerial Process*. McGraw Hill.
- Lattanze, A.J. (2008). *Architecting Software Intensive Systems: A Practitioner's Guide*.

CRC Press.
- Mathur, A.P. (2013). *Foundations of Software Testing (2nd ed.)*. Pearson Education.
- Simon, H.A. (2014). *A Student's Introduction to Engineering Design: Pergamon Unified Engineering Series (Vol. 21)*. Elsevier.

Acknowledgment